

Lab8

组员：杨桑多杰，张磊，徐南海

练习0：填写已有实验

本实验依赖实验2/3/4/5/6/7。请把你做的实验2/3/4/5/6/7的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”/“LAB6”/“LAB7”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab8的测试应用程序，可能需对已完成的实验2/3/4/5/6/7的代码进行进一步改进。

练习1：完成读文件操作的实现（需要编码）

首先了解打开文件的处理流程，然后参考本实验后续的文件读写操作的过程分析，填写在 kern/fs/sfs/sfs_inode.c 中的 sfs_io_nolock() 函数，实现读文件中数据的代码。

解答：

ucore 的文件系统模型源于 Havard 的 OS161 的文件系统和 Linux 文件系统。但其实这二者都是源于传统的 UNIX 文件系统设计。UNIX 提出了四个文件系统抽象概念：文件(file)、目录项(dentry)、索引节点(inode)和安装点(mount point)。

- 文件: UNIX 文件中的内容可理解为是一有序字节 buffer,文件都有一个方便应用程序识别的文件名称(也称文件路径名)。典型的文件操作有读、写、创建和删除等。
- 目录项:目录项不是目录,而是目录的组成部分。在 UNIX 中目录被看作一种特定的文件,而目录项是文件路径中的一部分。如一个文件路径名是 “/test/testfile”,则包含的目录项为:根目录 “/”,目录 “test” 和文件 “testfile”,这三个都是目录项。一般而言,目录项包含目录项的名字(文件名或目录名)和目录项的索引节点(见下面的描述)位置。
- 索引节点: UNIX 将文件的相关元数据信息(如访问控制权限、大小、拥有者、创建时间、数据内容等等信息)存储在一个单独的数据结构中,该结构被称为索引节点。
- 安装点:在 UNIX 中,文件系统被安装在一个特定的文件路径位置,这个位置就是安装点。所有的已安装文件系统都作为根文件系统树中的叶子出现在系统中。安装点是一个起点,从安装点开始可以访问文件系统中的所有文件。

其中,文件和目录是给应用程序看到的一个抽象。

从 ucore 操作系统不同的角度来看, ucore 中的文件系统架构包含四类主要的数据结构, 它们分别是:

1. 超级块(SuperBlock),它主要从文件系统的全局角度描述特定文件系统的全局信息。它的作用范围是整个OS空间。
2. 索引节点(inode):它主要从文件系统的单个文件的角度它描述了文件的各种属性和数据所在位置。它的作用范围是整个OS空间。
3. 目录项(dentry):它主要从文件的文件路径的角度描述了文件路径中的特定目录。它的作用范围是整个 OS 空间。
4. 文件(file),它主要从进程的角度描述了一个进程在访问文件时需要了解的文件标识,文件读写的位置,文件引用情况等信息。它的作用范围是某一具体进程。

文件系统,会将磁盘上的文件(程序)读取到内存里面来,在用户空间里面变成进程去进一步执行或其他操作。通过一系列系统调用完成这个过程。

根据实验指导书,我们可以了解到, ucore 的文件系统架构主要由四部分组成:

- 通用文件系统访问接口层:该层提供了一个从用户空间到文件系统的标准访问接口。这一层访问接口让应用程序能够通过一个简单的接口获得 ucore 内核的文件系统服务。
- 文件系统抽象层:向上提供一个一致的接口给内核其他部分(文件系统相关的系统调用实现模块和其他内核功能模块)访问。向下提供一个抽象函数指针列表和数据结构来屏蔽不同文件系统的实现细节。
- Simple FS 文件系统层: 一个基于索引方式的简单文件系统实例。向上通过各种具体函数实现以对应文件系统抽象层提出的抽象函数。向下访问外设接口
- 外设接口层:向上提供 device 访问接口屏蔽不同硬件细节。向下实现访问各种具体设备驱动接口,比如 disk 设备接口/串口设备接口/键盘设备接口等

这里我们可以通过下图可以比较好的理解这四个部分的关系:



例如某一个应用程序需要操作文件(增删读写等),首先需要通过文件系统的通用文件系统访问接口层给用户空间提供的访问接口进入文件系统内部,接着由文件系统抽象层把访问请求转发给某一具体文件系统(比如 Simple FS 文件系统),然后再由具体文件系统把应用程序的访问请求转化为对磁盘上的 block 的处理请求,并通过外设接口层交给磁盘驱动例程来完成具体的磁盘操作。

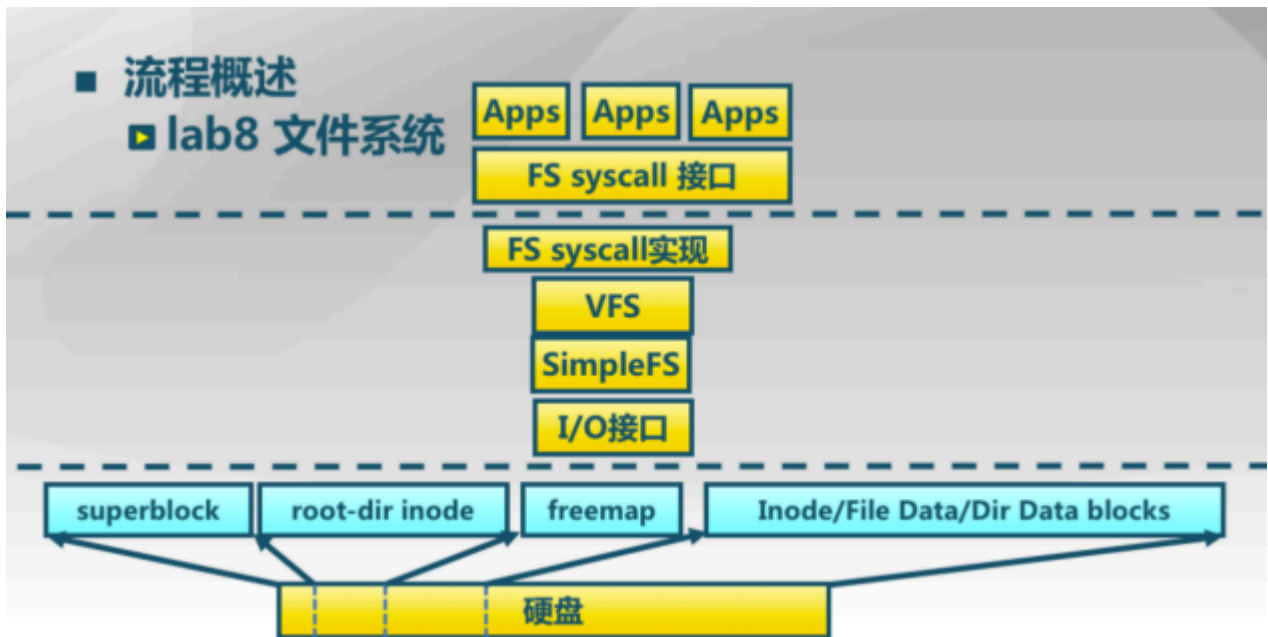
对应到我们的ucore上,具体的过程如下:

1. 以打开文件为例,首先用户会在进程中调用 safe_open() 函数,然后依次调用如下函数 open->sys_open->syscall,从而引发系统调用然后进入内核态,然后由 sys_open 内核函数处理系统调用,进一步调用到内核函数 sysfile_open,然后将字符串 "/test/testfile" 拷贝到内核空间中的字符串 path 中,并进入到文件系统抽象层的处理流程完成进一步的打开文件操作中。
2. 在文件系统抽象层,系统会分配一个 file 数据结构的变量,这个变量其实是 current->fs_struct->filemap[] 中的一个空元素,即还没有被用来打开过文件,但是分配完了之后还不能找到对应应的文件结点。所以系统在该层调用了 vfs_open 函数通过调用 vfs_lookup 找到 path 对应文件的 inode,然后调用vop_open函数打开文件。然后层

层返回,通过执行语句`file->node=node;`就把当前进程的 `current->fs_struct->filemap[fd]` (即 `file` 所指变量)的成员变量`node` 指针指向了代表文件的索引节点`node`。这时返回 `fd`。最后完成打开文件的操作。

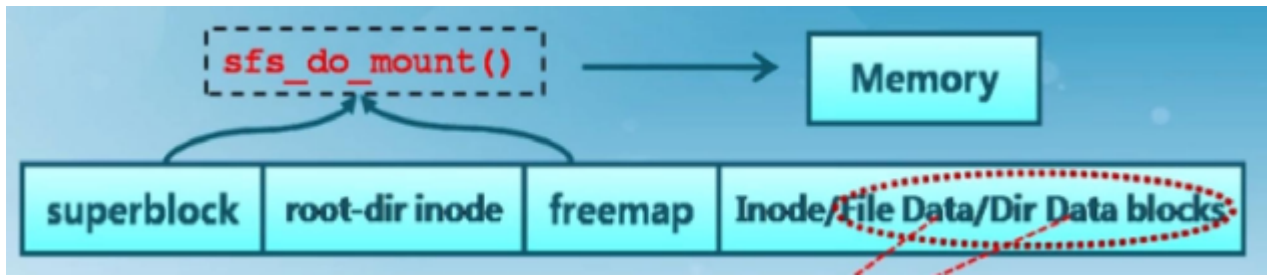
3. 在第2步中,调用了 SFS 文件系统层的 `vfs_lookup` 函数去寻找 `node`,这里在 `sfs_inode.c` 中我们能够知道 `vop_lookup = sfs_lookup`。
4. 看到 `sfs_lookup` 函数传入的三个参数,其中 `node` 是根目录`"/`所对应的 `inode` 节点; `path` 是文件的绝对路径(例如`"/test/file"`),而 `node_store` 是经过查找获得的 `file` 所对应的 `inode` 节点。函数以`"/`为分割符,从左至右逐一分解`path`获得各个子目录和最终文件对应的`inode` 节点。在本例中是分解出`"test"` 子目录,并调用 `sfs_lookup_once` 函数获得`"test"` 子目录对应的 `inode` 节点`subnode`,然后循环进一步调用 `sfs_lookup_once` 查找以`"test"` 子目录下的文件`"testfile1"` 所对应的 `inode` 节点。当无法分解 `path` 后,就意味着找到了`testfile1` 对应的 `inode` 节点,就可顺利返回了。
5. 而我们再进一步观察 `sfs_lookup_once` 函数,它调用 `sfs_dirent_search_nolock` 函数来查找与路径名匹配的目录项,如果找到目录项,则根据目录项中记录的 `inode` 所处的数据块索引值找到路径名对应的 SFS 磁盘 `inode`,并读入 SFS 磁盘 `inode` 对的内容,创建 SFS 内存 `inode`。

如下图所示, ucore 文件系统中,是这样处理读写硬盘操作的:



1. 首先是应用程序发出请求,请求硬盘中写数据或读数据,应用程序通过 FS syscall 接口执行系统调用,获得 ucore 操作系统关于文件的一些服务;
2. 之后,一旦操作系统内系统调用得到了请求,就会到达 VFS 层面(虚拟文件系统),包含很多部分比如文件接口、目录接口等,是一个抽象层面,它屏蔽底层具体的文件系统;
3. VFS 如果得到了处理,那么 VFS 会将这个 `iNode` 传递给 SimpleFS,注意,此时, VFS 中的 `iNode` 还是一个抽象的结构,在 SimpleFS 中会转化为一个具体的 `iNode`;
4. 通过该 `iNode` 经过 IO 接口对于磁盘进行读写。

那么,硬盘中的文件布局又是怎样的呢?硬盘中的布局信息存在SFS中,如下图所示:



上图所示的是一个 SFS 的文件系统,其定义在 (kern/fs/sfs/sfs.h , 83—94行):

```

struct sfs_fs {
    struct sfs_super super;           /* on-disk superblock */
    struct device *dev;               /* device mounted on */
    struct bitmap *freemap;           /* blocks in use are mared 0 */
    bool super_dirty;                 /* true if super/freemap modified */
    void *sfs_buffer;                 /* buffer for non-block aligned io */
    semaphore_t fs_sem;               /* semaphore for fs */
    semaphore_t io_sem;               /* semaphore for io */
    semaphore_t mutex_sem;            /* semaphore for link/unlink and rename */
    /*
    list_entry_t inode_list;           /* inode linked-list */
    list_entry_t *hash_list;           /* inode hash linked-list */
};

```

其中, SFS 的前 3 项对应的就是硬盘文件布局的全局信息。

那么,接下来分析这些文件布局的数据结构:

1、超级块 super_block (kern/fs/sfs/sfs.h , 40—45行)

```

struct sfs_super {
    uint32_t magic;                   /* magic number, should be SFS_MAGIC */
    uint32_t blocks;                  /* # of blocks in fs */
    uint32_t unused_blocks;           /* # of unused blocks in fs */
    char info[SFS_MAX_INFO_LEN + 1]; /* infomation for sfs */
};

```

超级块,刚刚说过是一个文件系统的全局角度描述特定文件系统的全局信息。这里面定义了标识符magic、总块数 blocks、空闲块数 unused_blocks 和一些关于 SFS 的信息,通常是字符串。

2、根目录结构 root_dir (kern/fs/sfs/sfs.h , 48—57行)

```

struct sfs_disk_inode {
    uint32_t size;                /* size of the file (in bytes) */
    uint16_t type;                /* one of SYS_TYPE_* above */
    uint16_t nlinks;              /* # of hard links to this file */
    uint32_t blocks;              /* # of blocks */
    uint32_t direct[SFS_NDIRECT]; /* direct blocks */
    uint32_t indirect;            /* indirect blocks */
    // uint32_t db_indirect;        /* double indirect blocks */
    // unused
};

```

我们刚刚讲过, iNode 是从文件系统的单个文件的角度它描述了文件的各种属性和数据所在位置,相当于一个索引,而 root_dir 是一个根目录索引,根目录表示,我们一开始访问这个文件系统可以看到的目录信息。主要关注 direct 和 indirect,代表根目录下的直接索引和间接索引。

3、目录项 entry (kern/fs/sfs/sfs.h , 60—63行)

```

/* file entry (on disk) */
struct sfs_disk_entry {
    uint32_t ino;                /* inode number */
    char name[SFS_MAX_FNAME_LEN + 1]; /* file name */
};

```

数组中存放的是文件的名字, ino 是该文件的 iNode 值。

仅有硬盘文件布局还不够, SFS 毕竟是一个在硬盘之上的抽象,它还需要传递上一层过来的索引值INODE。这个 INODE 是 SFS 层面的,我们刚刚讨论的 iNode 是硬盘上实际的索引。

4、sfs_inode (kern/fs/sfs/sfs.h , 69—77行)

```

/* inode for sfs */
struct sfs_inode {
    struct sfs_disk_inode *din; /* on-disk inode */
    uint32_t ino;                /* inode number */
    bool dirty;                  /* true if inode modified */
    int reclaim_count;           /* kill inode if it hits zero */
    semaphore_t sem;             /* semaphore for din */
    list_entry_t inode_link;      /* entry for linked-list in sfs_fs */
    list_entry_t hash_link;      /* entry for hash linked-list in sfs_fs */
};

```

我们看到, sfs_disk_inode 是 SFS 层面上的 iNode 的一个成员,代表了这两个结构之间的上下级关系。

接下来,我们分析更高层的数据结构 VFS (虚拟文件系统)。

在 VFS 层中,我们需要对于虚拟的 iNode,和下一层的 SFS 的 iNode 进行对接。

文件系统抽象层是把不同文件系统的对外共性接口提取出来,形成一个函数指针数组,这样,通用文件系统访问接口层只需访问文件系统抽象层,而不需关心具体文件系统的实现细节和接口。

1、VFS的抽象定义 (kern/fs/vfs/vfs.h , 35—46行)

```
struct fs {
    union {
        struct sfs_fs __sfs_info;
    } fs_info;                // filesystem-specific data
    enum {
        fs_type_sfs_info,
    } fs_type;                // filesystem type
    int (*fs_sync)(struct fs *fs); // Flush all dirty buffers to disk
    struct inode *(*fs_get_root)(struct fs *fs); // Return root inode of filesystem.
    int (*fs_unmount)(struct fs *fs); // Attempt unmount of filesystem.
    void (*fs_cleanup)(struct fs *fs); // Cleanup of filesystem.???
};
```

主要是一些函数指针用于处理 VFS 的操作。

2、文件结构 (kern/fs/file.h, 14—24行)

```
struct file {
    enum {
        FD_NONE, FD_INIT, FD_OPENED, FD_CLOSED,
    } status;
    bool readable;
    bool writable;
    int fd;
    off_t pos;
    struct inode *node;
    int open_count;
};
```

在 file 基础之上还有一个管理所有 file 的数据结构 file_struct (kern/fs/fs.h , 25—30行)

```
/*
 * process's file related information
 */
struct files_struct {
    struct inode *pwd; // inode of present working directory
    struct file *fd_array; // opened files array
    int files_count; // the number of opened files
    semaphore_t files_sem; // lock protect sem
};
```

3、VFS 的索引 iNode (kern/fs/vfs/inode.h , 29—42行)

```
struct inode {
    union {
```

```

    struct device __device_info;
    struct sfs_inode __sfs_inode_info;
} in_info;
enum {
    inode_type_device_info = 0x1234,
    inode_type_sfs_inode_info,
} in_type;
int ref_count;
int open_count;
struct fs *in_fs;
const struct inode_ops *in_ops;
};

```

我们看到在 VFS 层面的 iNode 值,包含了 SFS 和硬件设备 device 的情况。

4、inode 的操作函数指针列表 (kern/fs/vfs/inode.h, 169—186行)

```

struct inode_ops {
    unsigned long vop_magic;
    int (*vop_open)(struct inode *node, uint32_t open_flags);
    int (*vop_close)(struct inode *node);
    int (*vop_read)(struct inode *node, struct iobuf *iob);
    int (*vop_write)(struct inode *node, struct iobuf *iob);
    int (*vop_fstat)(struct inode *node, struct stat *stat);
    int (*vop_fsync)(struct inode *node);
    int (*vop_namefile)(struct inode *node, struct iobuf *iob);
    int (*vop_getdirent)(struct inode *node, struct iobuf *iob);
    int (*vop_reclaim)(struct inode *node);
    int (*vop_gettype)(struct inode *node, uint32_t *type_store);
    int (*vop_tryseek)(struct inode *node, off_t pos);
    int (*vop_truncate)(struct inode *node, off_t len);
    int (*vop_create)(struct inode *node, const char *name, bool excl, struct inode
**node_store);
    int (*vop_lookup)(struct inode *node, char *path, struct inode **node_store);
    int (*vop_ioctl)(struct inode *node, int op, void *data);
};

```

inode_ops 是对常规文件、目录、设备文件所有操作的一个抽象函数表示。对于某一具体的文件系统下的文件或目录,只需实现相关的函数,就可以被用户进程访问具体的文件了,且用户进程无需了解具体文件系统的实现细节。

有了上述分析后,我们可以看看如果一个用户进程打开文件会做些什么事情?

首先假定用户进程需要打开的文件已经存在在硬盘上。以 user/sfs_filetest1.c 为例,首先用户进程会调用在 main 函数中的如下语句:

```

int fd1 = safe_open("/test/testfile", O_RDWR | O_TRUNC);

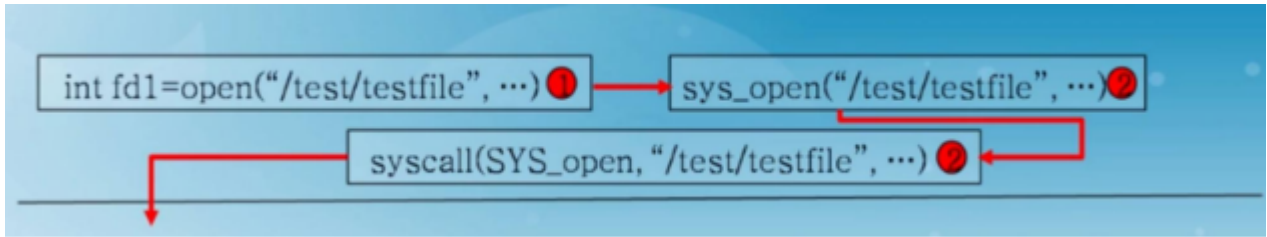
```

如果 ucore 能够正常查找到这个文件,就会返回一个代表文件的文件描述符 fd1,这样在接下来的读写文件过程中,就直接用这样 fd1 来代表就可以了。

接下来实现需要编码的函数:

通用文件访问接口层的处理流程:

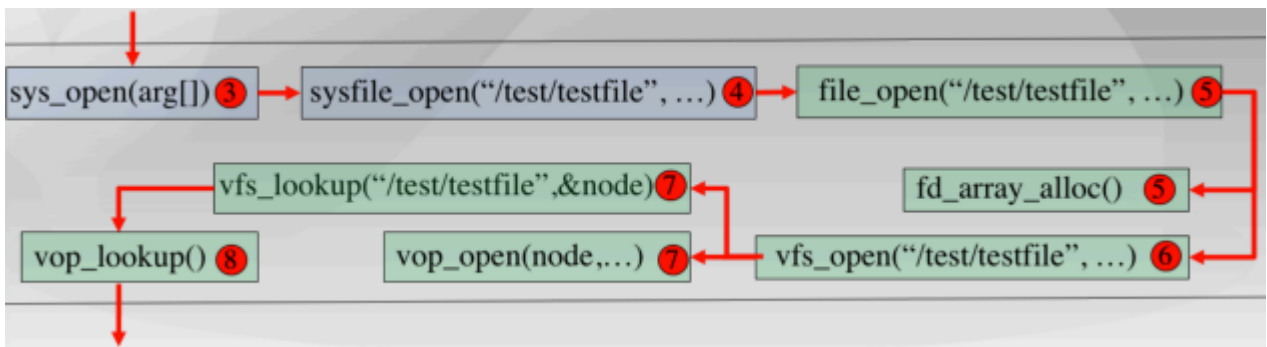
首先进入通用文件访问接口层的处理流程,即进一步调用如下用户态函数: open->sys_open->syscall,从而引起系统调用进入到内核态。到了内核态后,通过中断处理例程,会调用到 sys_open内核函数,并进一步调用 sysfile_open 内核函数。到了这里,需要把位于用户空间的字符串"/test/testfile" 拷贝到内核空间中的字符串 path 中,并进入到文件系统抽象层的处理流程完成进一步的打开文件操作中。



文件系统抽象层 (VFS)的处理流程:

1、分配一个空闲的 file 数据结构变量 file 在文件系统抽象层的处理中,首先调用的是file_open 函数,它要给这个即将打开的文件分配一个 file 数据结构的变量,这个变量其实是当前进程的打开文件数组 current->fs_struct->filemap[] 中的一个空闲元素(即还没用于一个打开的文件),而这个元素的索引值就是最终要返回到用户进程并赋值给变量 fd1。到了这一步还仅仅是给当前用户进程分配了一个file数据结构的变量,还没有找到对应的文件索引节点。

为此需要进一步调用 vfs_open 函数来找到 path 指出的文件所对应的基于 inode 数据结构的 VFS 索引节点 node。vfs_open 函数需要完成两件事情:通过 vfs_lookup 找到 path 对应文件的 inode;调用vop_open 函数打开文件。



2、找到文件设备的根目录/的索引节点需要注意,这里的 vfs_lookup 函数是一个针对目录的操作函数,它会调用 vop_lookup 函数来找到 SFS 文件系统下的 /test 目录下的 testfile 文件。为此,vfs_lookup 函数首先调用 get_device 函数,并进一步调用 vfs_get_bootfs 函数(其实调用了)来找到根目录/对应的 inode。这个 inode 就是位于vfs.c 中的 inode 变量 bootfs_node。这个变量在init_main 函数(位于kern/process/proc.c)执行时获得了赋值。

找到根目录/下的test子目录对应的索引节点,在找到根目录对应的inode后,通过调用vop_lookup函数来查找/和test这两层目录下的文件testfile所对应的索引节点,如果找到就返回此索引节点。

3、把 file 和 node 建立联系。完成第3步后,将返回到file_open 函数中,通过执行语句 file->node=node,就把当前进程的current->fs_struct->filemap[fd] (即file所指变量)的成员变量 node指针指向了代表 /test/testfile 文件的索引节点 node。这时返回 fd。经过重重回退,通过系统调用返回,用户态的 syscall->sys_open->open->safe_open 等用户函数的层层函数返回,最终把fd赋值给 fd1。自此完成了打开文件操作。但这里我们还没有分析第2和第3步是如何进一步调用 SFS 文件系统提供的函数找位于 SFS 文件系统上的 /test/testfile 所对应的 sfs 磁盘 inode 的过程。下面需要进一步对此进行分析。

sfs_lookup (kern/fs/sfs/sfs_inode.c , 996—1015行)

```
static int
sfs_lookup(struct inode *node, char *path, struct inode **node_store) {
```



```

    struct sfs_fs *sfs = fsop_info(vop_fs(node), sfs);
    assert(*path != '\0' && *path != '/');
    //以“/”为分割符,从左至右分解path获得各子目录和最终文件对应的inode节点。
    vop_ref_inc(node);
    struct sfs_inode *sin = vop_info(node, sfs_inode);
    if (sin->din->type != SFS_TYPE_DIR) {
        vop_ref_dec(node);
        return -E_NOTDIR;
    }
    struct inode *subnode;
    int ret = sfs_lookup_once(sfs, sin, path, &subnode, NULL); //循环进一步调用sfs_lookup_once
    查找以“test”子目录下的文件“testfile1”所对应的inode节点。

    vop_ref_dec(node);
    if (ret != 0) {
        return ret;
    }
    *node_store = subnode;
    //当无法分解path后,就意味着找到了需要对应的inode节点,就可顺利返回了。
    return 0;
}

```

看到函数传入的三个参数,其中 node 是根目录 “/” 所对应的 inode 节点; path 是文件的绝对路径(例如 “/test/file”),而 node_store 是经过查找获得的file所对应的inode节点。

函数以 “/” 为分割符,从左至右逐一分解path获得各个子目录和最终文件对应的inode 节点。在本例中是分解出 “test” 子目录,并调用sfs_lookup_once函数获得“test”子目录对应的 inode 节点 subnode ,然后循环进一步调用 sfs_lookup_once 查找以 “test” 子目录下的文件 “testfile1” 所对应的 inode 节点。当无法分解 path 后,就意味着找到了 testfile1 对应的 inode 节点,就可顺利返回了。

而我们再进一步观察 sfs_lookup_once 函数,它调用 sfs_dirent_search_nolock 函数来查找与路径名匹配的目录项,如果找到目录项,则根据目录项中记录的 inode 所处的数据块索引值找到路径名对应的 SFS 磁盘 inode,并读入 SFS 磁盘 inode 对的内容,创建 SFS 内存 inode。

sfs_lookup_once (kern/fs/sfs/sfs_inode.c , 498—512行)

```

static int
sfs_lookup_once(struct sfs_fs *sfs, struct sfs_inode *sin, const char *name, struct inode
**node_store, int *slot) {
    int ret;
    uint32_t ino;
    lock_sin(sin);
    { // find the NO. of disk block and logical index of file entry
        ret = sfs_dirent_search_nolock(sfs, sin, name, &ino, slot, NULL);
    }
    unlock_sin(sin);
    if (ret == 0) {
        // load the content of inode with the the NO. of disk block
        ret = sfs_load_inode(sfs, node_store, ino);
    }
    return ret;
}

```

最后是需要实现的函数:

```
static int
sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, void *buf, off_t offset, size_t
*alenp, bool write) {
    struct sfs_disk_inode *din = sin->din;
    assert(din->type != SFS_TYPE_DIR);
    off_t endpos = offset + *alenp, blkoff;
    *alenp = 0;
    // calculate the Rd/wr end position
    if (offset < 0 || offset >= SFS_MAX_FILE_SIZE || offset > endpos) {
        return -E_INVALID;
    }
    if (offset == endpos) {
        return 0;
    }
    if (endpos > SFS_MAX_FILE_SIZE) {
        endpos = SFS_MAX_FILE_SIZE;
    }
    if (!write) {
        if (offset >= din->size) {
            return 0;
        }
        if (endpos > din->size) {
            endpos = din->size;
        }
    }
}

int (*sfs_buf_op)(struct sfs_fs *sfs, void *buf, size_t len, uint32_t blkno, off_t
offset);
int (*sfs_block_op)(struct sfs_fs *sfs, void *buf, uint32_t blkno, uint32_t nblks);
if (write) {
    sfs_buf_op = sfs_wbuf, sfs_block_op = sfs_wblock;
}
else {
    sfs_buf_op = sfs_rbuf, sfs_block_op = sfs_rblock;
}

int ret = 0;
size_t size, alen = 0;
uint32_t ino;
uint32_t blkno = offset / SFS_BLKSIZE;          // The NO. of Rd/wr begin block
uint32_t nblks = endpos / SFS_BLKSIZE - blkno;  // The size of Rd/wr blocks

//LAB8:EXERCISE1 YOUR CODE HINT: call sfs_bmap_load_nolock, sfs_rbuf, sfs_rblock,etc. read
different kind of blocks in file
/*
    * (1) If offset isn't aligned with the first block, Rd/wr some content from offset to
the end of the first block
    *     NOTICE: useful function: sfs_bmap_load_nolock, sfs_buf_op
    *     Rd/wr size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset)
    * (2) Rd/wr aligned blocks
```

```

*      NOTICE: useful function: sfs_bmap_load_nolock, sfs_block_op
* (3) If end position isn't aligned with the last block, Rd/wr some content from begin
to the (endpos % SFS_BLKSIZE) of the last block
*      NOTICE: useful function: sfs_bmap_load_nolock, sfs_buf_op
*/

// (1) 处理第一个块的未对齐部分
if ((blkoff = offset % SFS_BLKSIZE) != 0) {
    // 计算需要读写的大小：如果有多个块，读到第一个块的末尾；否则读到endpos
    size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
    // 获取第一个块的磁盘块号
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    // 从块的blkoff位置开始读写size字节的数据
    if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
        goto out;
    }
    // 更新已处理的数据量和缓冲区指针
    alen += size;
    // 如果nblks为0，说明只有一个块，且已处理完，可以直接跳到out
    if (nblks == 0) {
        goto out;
    }
    buf += size;
    blkno++;
    nblks--;
}

// (2) 处理中间对齐的完整块
if (nblks > 0) {
    // 逐块处理（也可以一次处理多个块，但逐块处理更安全）
    for (uint32_t i = 0; i < nblks; i++) {
        // 获取当前块的磁盘块号
        if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
            goto out;
        }
        // 读写整个块
        if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
            goto out;
        }
        // 更新已处理的数据量和缓冲区指针
        alen += SFS_BLKSIZE;
        buf += SFS_BLKSIZE;
        blkno++;
    }
}

// (3) 处理最后一个块的未对齐部分
if ((size = endpos % SFS_BLKSIZE) != 0) {
    // 获取最后一个块的磁盘块号
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

```

```

    }
    // 从块的开始位置读写size字节的数据
    if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
        goto out;
    }
    // 更新已处理的数据量
    alen += size;
}

out:
    *alenp = alen;
    if (offset + alen > sin->din->size) {
        sin->din->size = offset + alen;
        sin->dirty = 1;
    }
    return ret;
}

```

每次通过 `sfs_bmap_load_nolock` 函数获取文件索引编号,然后调用 `sfs_buf_op` 完成实际的文件读写操作。

练习2: 完成基于文件系统的执行程序机制的实现（需要编码）

改写 `proc.c` 中的 `load_icode` 函数和其他相关函数，实现基于文件系统的执行程序机制。执行：make qemu。如果能看到 `sh` 用户程序的执行界面，则基本成功了。如果在 `sh` 用户界面上可以执行 `exit`, `hello`（更多用户程序放在 `user` 目录下）等其他放置在 `sfs` 文件系统系统中的其他执行程序，则可以认为本实验基本成功。

解答：

在 `proc.c` 中,根据注释我们需要先初始化 `fs` 中的进程控制结构,即在 `alloc_proc` 函数中我们需要做一下修改,加上一句 `proc->filesp = NULL`; 从而完成初始化。

为什么要这样做的呢,因为我们之前讲过, 一个文件需要在 `VFS` 中变为一个进程才能被执行。

修改之后 `alloc_proc` 函数如下:

```

// LAB8: 初始化文件系统相关字段
proc->filesp = NULL;

```

所以完整的 `alloc_proc` 函数的实现如下:

```

// alloc_proc - alloc a proc_struct and init all fields of proc_struct
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        proc->state = PROC_UNINIT;
    }
}

```

```

    proc->pid = -1;
    proc->runs = 0;
    proc->kstack = 0;
    proc->need_resched = 0;
    proc->parent = NULL;
    proc->mm = NULL;
    memset(&(proc->context), 0, sizeof(struct context));
    proc->tf = NULL;
    proc->pgdir = boot_pgdir_pa;
    proc->flags = 0;
    memset(proc->name, 0, PROC_NAME_LEN);
    // lab5 add:
    proc->wait_state = 0;
    proc->cptr = proc->optr = proc->yptr = NULL;
    proc->rq = NULL; // 初始化运行队列为空
    list_init(&(proc->run_link)); // 初始化运行队列的指针
    proc->time_slice = 0;
    proc->lab6_run_pool.left = proc->lab6_run_pool.right = proc->lab6_run_pool.parent =
NULL;

    proc->lab6_stride = 0;
    proc->lab6_priority = 0;

    // LAB8: 初始化文件系统相关字段
    proc->fileesp = NULL;
}
return proc;
}

```

此外 参数在栈中的布局如下所示:



```

    High Address |
    -----
    | Argument   |
    |   n       |
    |-----|
    |   ...     |
    |-----|
    | Argument   |
    |   1       |
    |-----|
    | padding    |
    |-----|
    | null ptr   |
    |-----|
    | Ptr Arg n  |
    |-----|
    |   ...     |
    |-----|
    | Ptr Arg 1  |
    |-----|
    | Arg Count  | <-- user esp
    -----

```

然后就是要实现proc_run 函数,具体的实现及注释如下所示:

```

void proc_run(struct proc_struct *proc)
{
    // LAB4:填写你在lab4中实现的代码
    /*
     * Some Useful MACROS, Functions and DEFINES, you can use them in below
     implementation.
     * MACROS or Functions:
     *   local_intr_save():      Disable interrupts
     *   local_intr_restore():   Enable Interrupts
     *   lcr3():                  Modify the value of CR3 register
     *   switch_to():             Context switching between two processes
     */
    bool intr_flag;
    local_intr_save(intr_flag);
    struct proc_struct *prev = current;
    current = proc;
    lcr3(current->pgdir);

    //LAB8 YOUR CODE : (update LAB4 steps)
    /*
     * below fields(add in LAB6) in proc_struct need to be initialized
     * before switch_to();you should flush the tlb
     * MACROS or Functions:
     *   flush_tlb():             flush the tlb
     */
    flush_tlb(); // 刷新TLB, 确保页表切换生效

    switch_to(&(prev->context), &(current->context));
    local_intr_restore(intr_flag);
}

```

实现do_fork函数,具体的实现及注释如下所示:

```

int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;
    proc = alloc_proc();
    if (proc == NULL)
    {
        goto fork_out;
    }
}

```



```

    }
    proc->parent = current;
    assert(current->wait_state == 0);
    if (setup_kstack(proc) != 0)
    {
        goto bad_fork_cleanup_proc;
    }
    if (copy_mm(clone_flags, proc) != 0)
    {
        goto bad_fork_cleanup_kstack;
    }

    // LAB8: 复制文件系统信息
    if (copy_files(clone_flags, proc) != 0)
    {
        goto bad_fork_cleanup_mm;
    }

    copy_thread(proc, stack, tf);

    bool intr_flag;
    local_intr_save(intr_flag);
    {
        proc->pid = get_pid();
        hash_proc(proc);
        set_links(proc);
    }
    local_intr_restore(intr_flag);

    wakeup_proc(proc);
    ret = proc->pid;

fork_out:
    return ret;

bad_fork_cleanup_fs: // for LAB8
    put_files(proc);
bad_fork_cleanup_mm:
    exit_mmap(proc->mm);
    put_pgdir(proc->mm);
    mm_destroy(proc->mm);
bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

实现load_icode函数,具体的实现及注释如下所示:

```


static int

```

```

load_icode(int fd, int argc, char **kargv)
{
    /* LAB8:EXERCISE2 YOUR CODE HINT:how to load the file with handler fd in to process's
memory? how to setup argc/argv?
    * MACROS or Functions:
    * mm_create          - create a mm
    * setup_pgdir        - setup pgdir in mm
    * load_icode_read    - read raw data content of program file
    * mm_map             - build new vma
    * pgdir_alloc_page   - allocate new memory for TEXT/DATA/BSS/stack parts
    * lsatp              - update Page Directory Addr Register -- CR3
    */
    //You can Follow the code form LAB5 which you have completed to complete
    /* (1) create a new mm for current process
    * (2) create a new PDT, and mm->pgdir= kernel virtual addr of PDT
    * (3) copy TEXT/DATA/BSS parts in binary to memory space of process
    * (3.1) read raw data content in file and resolve elfhdr
    * (3.2) read raw data content in file and resolve proghdr based on info in elfhdr
    * (3.3) call mm_map to build vma related to TEXT/DATA
    * (3.4) callpgdir_alloc_page to allocate page for TEXT/DATA, read contents in file
    * and copy them into the new allocated pages
    * (3.5) callpgdir_alloc_page to allocate pages for BSS, memset zero in these pages
    * (4) call mm_map to setup user stack, and put parameters into user stack
    * (5) setup current process's mm, cr3, reset pgidr (using lsatp MARCO)
    * (6) setup uargc and uargv in user stacks
    * (7) setup trapframe for user environment
    * (8) if up steps failed, you should cleanup the env.
    */

    assert(argc >= 0 && argc <= EXEC_MAX_ARG_NUM);

    // (1) 创建新的mm结构
    if (current->mm != NULL) {
        panic("load_icode: current->mm must be empty.\n");
    }

    int ret = -E_NO_MEM;
    struct mm_struct *mm;
    if ((mm = mm_create()) == NULL) {
        goto bad_mm;
    }

    // (2) 创建新的页目录表PDT
    if (setup_pgdir(mm) != 0) {
        goto bad_pgdir_cleanup_mm;
    }

    struct Page *page;

    // (3) 从ELF文件中加载程序段
    // (3.1) 读取ELF头
    struct elfhdr __elf, *elf = &__elf;
    if ((ret = load_icode_read(fd, elf, sizeof(struct elfhdr), 0)) != 0) {

```

```

        goto bad_elf_cleanup_pgdir;
    }

    // 检查ELF魔数
    if (elf->e_magic != ELF_MAGIC) {
        ret = -E_INVALID ELF;
        goto bad_elf_cleanup_pgdir;
    }

    // (3.2) 读取程序头表
    struct proghdr __ph, *ph = &__ph;
    uint64_t ph_off, ph_end;

    // 遍历所有程序头
    for (int i = 0; i < elf->e_phnum; i++) {
        ph_off = elf->e_phoff + sizeof(struct proghdr) * i;
        if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr), ph_off)) != 0) {
            goto bad_cleanup_mmap;
        }

        // 只处理LOAD类型的段
        if (ph->p_type != ELF_PT_LOAD) {
            continue;
        }
        if (ph->p_filesz > ph->p_memsz) {
            ret = -E_INVALID ELF;
            goto bad_cleanup_mmap;
        }
        if (ph->p_filesz == 0) {
            continue; // 跳过空段
        }

        // (3.3) 建立vma
        uint32_t vm_flags = 0;
        uint32_t perm = PTE_U | PTE_V;
        if (ph->p_flags & ELF_PF_X) vm_flags |= VM_EXEC;
        if (ph->p_flags & ELF_PF_W) vm_flags |= VM_WRITE;
        if (ph->p_flags & ELF_PF_R) vm_flags |= VM_READ;

        // 如果段可写，设置PTE_W标志
        if (vm_flags & VM_WRITE) perm |= (PTE_R | PTE_W);
        else if (vm_flags & VM_READ) perm |= PTE_R;
        if (vm_flags & VM_EXEC) perm |= PTE_X;

        // 段的虚拟地址范围
        uintptr_t vm_start = ph->p_va;
        uintptr_t vm_end = ph->p_va + ph->p_memsz;

        // 建立vma结构
        if ((ret = mm_map(mm, vm_start, vm_end - vm_start, vm_flags, NULL)) != 0) {
            goto bad_cleanup_mmap;
        }
    }

```

```

// (3.4) 为TEXT/DATA分配页面并复制内容
uintptr_t start = vm_start, end, la;
off_t offset = ph->p_offset;
size_t off, size;

// 计算第一个页的偏移
off = start - ROUNDDOWN(start, PGSIZE);
size = PGSIZE - off;
la = ROUNDDOWN(start, PGSIZE);

// 复制文件内容到内存
end = ph->p_va + ph->p_filesz;
while (start < end) {
    // 分配页面
    if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
        ret = -E_NO_MEM;
        goto bad_cleanup_mmap;
    }

    // 计算本次要复制的大小
    if (size > end - start) {
        size = end - start;
    }

    // 从文件读取数据到页面
    if ((ret = load_icode_read(fd, page2kva(page) + off, size, offset)) != 0) {
        goto bad_cleanup_mmap;
    }

    // 更新偏移和地址
    start += size;
    offset += size;
    off = 0;
    size = PGSIZE;
    la += PGSIZE;
}

// (3.5) 为BSS段分配页面并清零
// BSS段从p_filesz到p_memsz
end = ph->p_va + ph->p_memsz;

// 如果当前页还有剩余空间，先清零
if (start % PGSIZE != 0) {
    // 清零当前页的剩余部分
    size = PGSIZE - (start % PGSIZE);
    if (start + size > end) {
        size = end - start;
    }
    memset(page2kva(page) + (start % PGSIZE), 0, size);
    start += size;
    la += PGSIZE;
}

```

```

// 为剩余的BSS段分配页面
while (start < end) {
    if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
        ret = -E_NO_MEM;
        goto bad_cleanup_mmap;
    }

    size = PGSIZE;
    if (start + size > end) {
        size = end - start;
    }
    memset(page2kva(page), 0, size);
    start += size;
    la += PGSIZE;
}
}

// 关闭文件
sysfile_close(fd);

// (4) 设置用户栈
uint32_t vm_flags_stack = VM_READ | VM_WRITE | VM_STACK;
if ((ret = mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, vm_flags_stack, NULL)) != 0) {
    goto bad_cleanup_mmap;
}

// 为用户栈分配内存
assert(USTACKTOP % PGSIZE == 0);
uintptr_t ustacktop = USTACKTOP;
for (int i = 0; i < USTACKPAGE; i++) {
    if ((page = pgdir_alloc_page(mm->pgdir, ustacktop - PGSIZE * (i + 1), PTE_U | PTE_V
| PTE_R | PTE_W)) == NULL) {
        ret = -E_NO_MEM;
        goto bad_cleanup_mmap;
    }
}

// (5) 设置mm, cr3
mm_count_inc(mm);
current->mm = mm;
current->pgdir = PADDR(mm->pgdir);
lsatp(PADDR(mm->pgdir));

// (6) 设置用户栈中的argc和argv
// 将参数压入用户栈
uintptr_t stacktop = USTACKTOP;

// 首先计算所有参数字符串的总长度
size_t total_len = 0;
for (int i = 0; i < argc; i++) {
    total_len += strlen(kargv[i]) + 1; // +1 for '\0'
}

```

```

// 在栈上为字符串分配空间（先放字符串）
stacktop -= total_len;
stacktop = ROUNDDOWN(stacktop, sizeof(uintptr_t)); // 对齐

// 复制参数字符串到用户栈
char **uargv = (char **)(stacktop - sizeof(char *) * (argc + 1)); // +1 for NULL
terminator

uintptr_t str_ptr = stacktop;
for (int i = 0; i < argc; i++) {
    size_t len = strlen(kargv[i]) + 1;
    strcpy((char *)str_ptr, kargv[i]);
    uargv[i] = (char *)str_ptr;
    str_ptr += len;
}
uargv[argc] = NULL;

// 栈顶指针移动到argv数组之前
stacktop = (uintptr_t)uargv;
stacktop -= sizeof(uintptr_t); // 为argv留空间
*(uintptr_t *)stacktop = argc;

// 再次对齐栈指针
stacktop = ROUNDDOWN(stacktop, 16); // RISC-V要求16字节对齐

// (7) 设置trapframe
struct trapframe *tf = current->tf;
memset(tf, 0, sizeof(struct trapframe));

tf->gpr.sp = stacktop; // 设置用户栈指针
tf->epc = elf->e_entry; // 设置程序入口点
tf->status = read_csr(sstatus) & ~SSTATUS_SPP; // 用户态，清除SPP位
tf->status |= SSTATUS_SPIE; // 使能中断

ret = 0;

out:
    return ret;

bad_cleanup_mmap:
    exit_mmap(mm);
bad_elf_cleanup_pgdir:
    put_pgdir(mm);
bad_pgdir_cleanup_mm:
    mm_destroy(mm);
bad_mm:
    goto out;
}

```

load_icode 主要是将文件加载到内存中执行,从上面的注释可知分为了一共七个步骤:

- 建立内存管理器
- 建立页目录

- 将文件逐个段加载到内存中,这里要注意设置虚拟地址与物理地址之间的映射
- 建立相应的虚拟内存映射表
- 建立并初始化用户堆栈
- 处理用户栈中传入的参数
- 最后很关键的一步是设置用户进程的中断帧
- 发生错误还需要进行错误处理。

当然一旦发生错误还需要进行错误处理。

实验结果

实验结果如图所示

```
rowin@Rowin:~/Documents/oslab/lab8$ make qemu
```

```
PMP0: 0x00000000080000000-0x0000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
(THU.CST) os is loading ...
```

```
Special kernel symbols:
```

```
entry 0xc020004a (virtual)
etext 0xc020b48c (virtual)
edata 0xc0291060 (virtual)
end    0xc0296910 (virtual)
```

```
Kernel executable memory footprint: 603KB
```

```
DTB Init
```

```
HartID: 0
```

```
DTB Address: 0x82200000
```

```
Physical Memory from DTB:
```

```
Base: 0x0000000008000000
Size: 0x0000000008000000 (128 MB)
End: 0x00000000087fffffff
```

```
DTB init completed
```

```
memory management: default_pmm_manager
```

```
physical memory map:
```

```
memory: 0x08000000, [0x80000000, 0x87fffffff].
```

```
vapaofset is 18446744070488326144
```

```
check_alloc_page() succeeded!
```

```
Page table directory switch succeeded!
```

```
Kernel stack guardians set succeeded!
```

```
check_pgdir() succeeded!
```

```
check_boot_pgdir() succeeded!
```

```
use SLOB allocator
```

```
kmalloc_init() succeeded!
```

```
check_vma_struct() succeeded!
```

```
check_vmm() succeeded.
```

```
sched class: RR_scheduler
```

```
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
```

```
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
```

```
sfs: mount: 'simple file system' (106/11/117)
```

```
vfs: mount disk0.
```

```
++ setup timer interrupts
```

```
kernel_execve: pid = 2, name = "sh".
```

```
user sh is running!!!
```

```
Hello world!!.
```

```
I am process 3.
```

```
hello pass.
```

```
$
```